

Bloque 02 - Python desde cero

Propósito

Aprender a leer, escribir y depurar programas pequeños antes de usar bibliotecas de análisis. El objetivo no es memorizar sintaxis: es expresar una regla de negocio de forma clara y verificable.

Lecciones

1. Ejecutar código, valores y variables
2. Listas, diccionarios y datos sencillos
3. Decisiones y repeticiones
4. Funciones y alcance
5. Errores y depuración
6. Estilo y práctica guiada

Prerrequisitos

Haber leído los bloques 00 y 01. Basta con saber que un dato debe tener significado y que una regla analítica debe poder revisarse.

Práctica

Abre el [notebook de gastos personales](#) o [ejecútalo en Google Colab](#). También puedes resolver el [ejercicio](#). Las [soluciones](#) se consultan al terminar.

Ejecutar código, valores y variables

Objetivos y prerrequisitos

Aprenderás qué hace un programa, cómo ejecutar una celda de notebook y cómo guardar un resultado con un nombre. No necesitas experiencia previa.

Código que transforma valores

Un programa es una lista de instrucciones que transforma información. En un análisis, esa información puede ser el importe de una compra, una fecha o una respuesta de usuario. Un **valor** es una pieza concreta de información: 1200, "web" o True (verdadero).

En Google Colab, un notebook muestra una celda de código y su resultado. Ejecuta primero esto y cambia después un único valor:

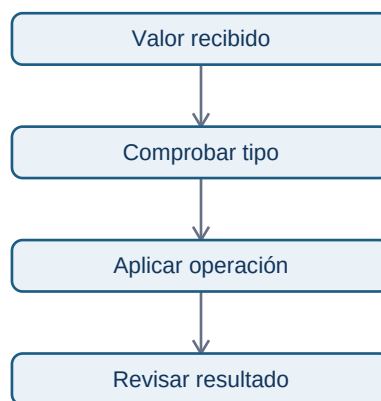
```
ventas = 1200
objetivo = 1500
cumplimiento = ventas / objetivo
print(cumplimiento)
```

Una **variable** es un nombre que referencia un valor. Aquí cumplimiento guarda el resultado 0.8. El signo = no pregunta si dos cosas son iguales: asigna el valor de la derecha al nombre de la izquierda.

Tipos: la forma del dato importa

Python distingue números enteros (3), decimales (3.5), texto ("Madrid"), booleanos (True o False) y ausencia representada por None. El tipo condiciona qué operaciones tienen sentido: sumar 3 + 5 es válido; sumar "3" + "5" une texto y produce "35".

Esta secuencia responde a “¿por qué revisar el tipo antes de calcular?”



El paso de comprobar evita, por ejemplo, tratar un importe escrito como texto como si fuera dinero calculable.

Error habitual

ventas = ventas + 100 parece una igualdad matemática imposible. En programación significa “toma el valor actual de ventas, súmale 100 y guarda el nuevo resultado bajo el mismo nombre”. Usarlo sin cuidado puede ocultar el valor original; cuando importe, conserva ambos nombres:

ventas_iniciales y ventas_actualizadas.

Resumen y comprobación

- Una celda ejecuta instrucciones y muestra un resultado.
- Una variable etiqueta un valor; no es una caja mágica independiente.
- El tipo determina qué cálculo es válido.

Prueba `type(ventas)` y `type("1200")`. Explica por qué se diferencian. Continúa con [colecciones](#).

Listas, diccionarios y datos sencillos

Objetivos y prerequisites

Sabrás agrupar varios valores y representar una compra sencilla antes de conocer las tablas de Pandas. Requiere variables y tipos básicos.

Cuando un valor no basta

Una lista guarda una secuencia ordenada. Por ejemplo, los importes de tres pedidos:

```
importes = [12.50, 18.00, 7.20]
primer_importe = importes[0]
```

Los corchetes indican una **lista** y el índice empieza en cero: `importes[0]` es el primer elemento. Esto sorprende al principio, así que no intentes “corregirlo”: compruébalo imprimiendo el resultado.

Un **diccionario** asocia una clave con un valor. Es útil para una observación con campos nombrados:

```
pedido = {"canal": "web", "importe": 42.50, "pagado": True}
print(pedido["importe"])
```

La clave evita depender de una posición. Una lista de diccionarios puede representar varias compras pequeñas; más adelante Pandas convertirá esa estructura en una tabla.

Relación con datos reales

Una API —un mecanismo para que programas intercambien información— suele devolver listas y diccionarios similares. Eso no elimina la necesidad de validar: que un campo se llame `importe` no garantiza que sea numérico, completo o esté expresado en la misma moneda.

Límite y práctica

`pedido["descuento"]` provoca un error si esa clave no existe. No inventes un cero sin preguntar qué significa que falte: podría significar “sin descuento”, “dato desconocido” o “campo no recogido”.

Resume con tus palabras la diferencia entre lista y diccionario y continúa con [condiciones y bucles](#).

Decisiones y repeticiones

Objetivos y prerequisites

Aplicarás una regla distinta según un dato y repetirás una comprobación sobre varios pedidos. Requiere listas y diccionarios.

Condiciones: expresar una regla

Una condición permite que el programa elija. `if` pregunta por una expresión que da `True` o `False`:

```
importe = 120
if importe >= 100:
    categoria = "pedido alto"
else:
    categoria = "pedido habitual"
```

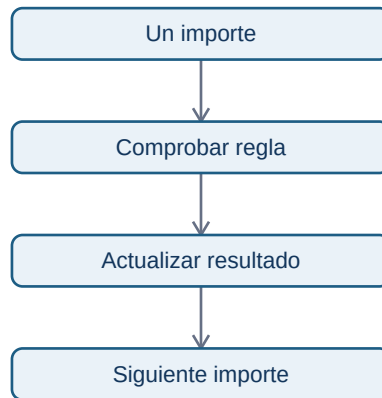
La sangría no es decoración: las líneas desplazadas pertenecen a la rama de la condición. La regla debe coincidir con una definición de negocio. Pregunta si exactamente 100 debe contarse como alto antes de decidir entre `>` y `>=`.

Bucles: repetir sin copiar código

Un bucle `for` visita cada elemento de una colección. El siguiente acumula importes y muestra la idea de agregación que luego hará Pandas:

```
total = 0
for importe in [12.5, 18.0, 7.2]:
    total = total + importe
print(total)
```

Este flujo responde a “¿qué ocurre para cada dato de entrada?”



El programa repite el mismo criterio; no decide de forma inteligente qué regla usar. La calidad de la conclusión depende de que la regla y los datos sean apropiados.

Error habitual

No modifiques una lista mientras la recorres salvo que entiendas las consecuencias: puedes saltarte elementos. Para aprender, crea una lista nueva para los resultados o revisa primero el tamaño y contenido de la original.

Resumen

Las condiciones expresan criterios y los bucles los aplican repetidamente. Sigue con [funciones](#).

Funciones y alcance

Objetivos y prerequisites

Aprenderás a encapsular una regla reutilizable, diferenciar entrada y salida y evitar depender de variables ocultas. Requiere condiciones y bucles.

Dar nombre a una transformación

Una **función** es un fragmento de código con un nombre, entradas y una salida. Evita copiar la misma regla en diez lugares y hace que el análisis se pueda revisar por partes.

```
def clasificar_importe(importe):  
    if importe >= 100:
```

```
        return "alto"
    return "habitual"

clasificar_importe(120)
```

importe es un **parámetro**: el nombre que la función usa para recibir un dato. `return` devuelve un resultado. Una función útil responde una pregunta concreta: “dado un importe, ¿qué etiqueta corresponde según esta regla?”.

Alcance: qué nombres existen dónde

Los nombres creados dentro de una función normalmente solo existen durante esa llamada. Esto se llama **alcance**. Es una protección: una función debería depender de sus entradas, no de una variable lejana cuyo valor puede cambiar sin avisar.

```
limite = 100

def es_alto(importe, limite):
    return importe >= limite
```

Aunque parece repetitivo pasar `limite`, deja visible el supuesto. En un análisis, los supuestos invisibles son difíciles de auditar.

Caso IT y límite

Una función puede estandarizar una comprobación de eventos: clasificar una duración de carga como lenta o aceptable. Pero no convierte el umbral en verdad universal: 2 segundos puede ser aceptable en una página y grave durante un pago. Documenta de dónde sale el umbral.

Resumen y práctica

- Las funciones nombran transformaciones repetibles.
- Parámetros hacen visibles las entradas; `return` entrega la salida.
- Evitar dependencias ocultas facilita revisar y probar.

Reescribe el cálculo de “pedido alto” como función y pruébalo con 99, 100 y 101. Después estudia [errores y depuración](#).

Errores y depuración

Objetivos y prerequisites

Sabrás leer el mensaje de error, formular una hipótesis mínima y comprobarla sin cambiar diez cosas a la vez. Requiere haber ejecutado código sencillo.

Un error también es evidencia

Cuando Python no puede continuar, muestra un mensaje con una última línea relevante. `NameError` suele indicar que un nombre no existe; `TypeError`, que se intentó una operación incompatible; `KeyError`, que falta una clave del diccionario. El mensaje no sustituye la comprensión, pero delimita qué revisar.

Ejemplo:

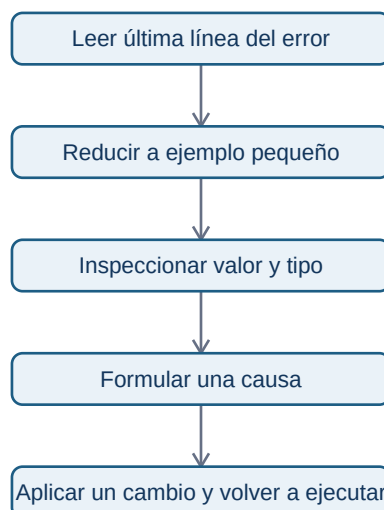
```
importe = "42.50"  
importe + 10
```

El problema no es que Python “falle”: `importe` contiene texto, no un número. La corrección solo debe hacerse si la regla de origen confirma que ese texto representa un importe válido:

```
importe_numerico = float(importe)
```

Método de depuración

Este flujo responde a “¿cómo investigar sin adivinar?”



Reducir el ejemplo evita mezclar varios problemas. Usa `print(valor)` y `type(valor)` antes de cambiar código. Si la causa es un dato inesperado, no la tapes con una conversión automática: registra cuántos casos hay y decide qué significan.

Error habitual: silenciar en lugar de entender

`try/except` permite manejar errores, pero capturarlos todos y continuar puede ocultar importes inválidos o pedidos perdidos. Úsalo para casos esperados y registra qué ocurrió. Un análisis correcto que descarta silenciosamente el 20 % de los datos no es fiable.

Resumen

Depurar es un proceso de evidencia: leer, aislar, inspeccionar, probar. Continúa con [estilo y práctica](#).

Estilo y práctica guiada

Objetivos y prerequisites

Aplicarás las piezas del bloque en un problema pequeño y aprenderás a escribir código que otra persona pueda leer. Requiere todas las lecciones anteriores.

Legibilidad es una propiedad analítica

Un programa de análisis no se evalúa solo por “funciona hoy”. Debe dejar claro qué representa cada valor y qué regla se aplicó. Usa nombres como `gasto_por_categoria`, no `x`; evita repetir números mágicos como 100 sin explicar su significado; separa carga de datos, transformación y resultado.

Un ejemplo legible:

```
LIMITE_REVISAR = 100

def categorias_sobre_limite(movimientos, limite):
    total_por_categoria = {}
    for movimiento in movimientos:
        categoria = movimiento["categoria"]
        importe = movimiento["importe"]
        total_por_categoria[categoria] = total_por_categoria.get(categoria, 0) + importe
    return [categoria for categoria, total in total_por_categoria.items() if total > limite]
```

Antes de reutilizarlo en una empresa, define cómo se tratan devoluciones, moneda, movimientos duplicados y valores ausentes. El código implementa una decisión; no decide por ti qué es correcto.

Comprobaciones mínimas

Prueba casos normales y casos límite: lista vacía, importe exactamente 100, importe negativo y un texto donde debería haber número. Escribir esos ejemplos antes de confiar en el resultado es una

forma simple de prueba.

Ejercicio de cierre

Resuelve la [práctica de gastos](#) y compara tu razonamiento con las [soluciones](#) solo al terminar. Si solo tienes móvil, escribe primero el pseudocódigo en texto: entradas, pasos y salidas.

Puente al siguiente bloque

Python permite operar con estructuras pequeñas. En NumPy y Pandas aplicarás operaciones parecidas a miles de valores y tablas, pero conservarás las mismas preguntas: ¿qué representa cada dato?, ¿qué regla se aplica?, ¿cómo se comprueba?